

AI Capstone: Milestone 2 Check-in

Project Name: MotionPlay — A Camera-Based Body-Motion Game Controller

Due: Week 6 Team: Kunal Pandey · Lei Li · Chao Chen · Chenghua Jiang

Focus: Demonstrating progress on data handling, feature engineering, and an initial working core-system prototype.

1. Data Status & Preparation

1.1 Data Sources

Two data streams are now processed through one shared pipeline, so that the pre-trained model and the gesture data share a single, identical representation. This consistency is the foundation of the whole approach: anything learned on the public dataset can transfer cleanly to our self-collected gestures.

- **AIST++ (Pre-training Source)**

- *Type/Role*: Public multi-view 3D dance dataset; used to pre-train the core motion model so it learns general full-body movement features.
- *Format*: COCO-17 three-dimensional keypoint sequences, one clip per file.
- *Volume*: 1,408 clips spanning 10 dance genres, all successfully processed with zero files skipped.

- **Self-Collected ARKit (Primary Source)**

- *Type/Role*: Primary dataset for the five-gesture recognition task, validation, and the final live demonstration.
- *Format*: Long-format CSV containing per-joint 3D positions and rotation quaternions across 91 tracked joints.
- *Status*: The parsing and preprocessing pipeline is complete and verified on a sample recording; full gesture collection is scheduled for the next phase.

1.2 Cleaning & Preprocessing Pipeline (Implemented)

A single shared pipeline transforms raw skeleton data from either source into a clean, normalized, fixed-length sequence. The four stages below are fully implemented and run on every clip:

1. **Joint Filtering**. Both the 91-joint ARKit skeleton and the 17-joint COCO AIST++ skeleton are reduced to a common 12-joint body subset (shoulders, elbows, wrists, hips, knees, and

ankles). Noisy facial and finger joints are stripped out entirely, and the two sources are forced to an identical graph topology so they become directly comparable.

2. **Spatial Normalization.** Each frame is centered on the hip midpoint (a virtual pelvis) and then scaled by the clip's mean torso length. This removes the player's absolute position in the room as well as differences in body height and measurement unit (AIST++ is recorded in millimetres, ARKit in metres).
3. **Orientation Alignment.** A first-frame canonical rotation removes each recording's arbitrary starting heading while deliberately preserving any turning that happens within the clip. This directly addresses the inconsistent global orientation observed across separate ARKit sessions in Milestone 1.
4. **Temporal Resampling.** Every clip is interpolated to a fixed length of 300 frames. This makes variable-length recordings comparable to one another and allows them to be stacked into training batches without padding.

1.3 Challenges Encountered & Resolved

Several practical issues surfaced while aligning two very different data sources. Each was diagnosed and resolved before moving on to modeling:

- **Coordinate-system mismatch.** AIST++ and ARKit live in independent world frames. We diagnosed the relationship by plotting each skeleton's hip-to-shoulder vector and confirmed both are Y-up, so no axis swap was required — the apparent misalignment seen in early overlays was simply two different poses, not a bug.
- **Unit and scale gap.** The millimetre-versus-metre difference is fully absorbed by the torso-length scaling, which is unit-invariant by construction.
- **Local versus world coordinates.** Using simple limb-length arithmetic we verified that the ARKit positions are already expressed in a common hip-rooted space, so no hierarchical transform accumulation is needed when reading the CSV.

2. Feature Engineering & Rationale

The model consumes the normalized 12-joint skeleton sequence as a spatio-temporal graph, where joints are vertices and bones are edges. The main engineering choices and the reasoning behind each are as follows:

- **12-Joint Graph Topology.** A hand-built adjacency matrix connects the limbs and closes the torso as a quadrilateral linking the shoulders and hips. A smaller set of reliable joints outperforms a large set that includes noisy facial and finger tracking, and it guarantees that both data sources use exactly the same graph.

- **Invariance by Design.** Hip-centering, torso-length scaling, and first-frame rotation together make the features invariant to where the player stands, how tall they are, and which direction they face. These are precisely the nuisance factors that differ from user to user and from recording to recording.
- **Data Augmentation.** During pre-training we apply random rotation about the vertical axis (up to $\pm 180^\circ$), left/right mirroring with correct swapping of left and right joints, and optional small Gaussian jitter. This artificially expands the dataset and counters the limited volume of manual samples flagged as a risk in Milestone 1.
- **Learnable Edge Importance.** Each graph-convolution layer multiplies the adjacency matrix by a learnable weight, allowing the network to emphasize the bone connections that matter most for distinguishing one motion from another.

3. Core Prototype — Runnable Demonstration

3.1 Model and Implementation

We brought forward the Stage-4 stretch-goal model, the Spatio-Temporal Graph Convolutional Network (ST-GCN), in order to validate the most ambitious branch of our plan early rather than at the very end. It is implemented from scratch in PyTorch in roughly 150 lines, comprising graph convolution over the 12-joint adjacency, temporal convolutions along the time axis, residual blocks, a global pooling layer that produces a fixed 256-dimensional feature vector, and a classification head.

3.2 What Runs Today

- **Task.** Ten-way dance-genre classification on the 1,408 AIST++ clips. This serves as a proxy task that forces the backbone to learn general full-body motion features, which is the real goal of pre-training.
- **Training.** The full training loop runs end-to-end on the GPU. The loss decreases smoothly and converges over 50 epochs without instability.

3.3 Initial Results

The table below summarizes the current training run:

Metric	Result	Interpretation
Train accuracy (10 genres)	~93%	Far above the 10% random baseline.
Final training loss	~0.23	Smooth, stable convergence with no divergence.

Processed clips	1,408 / 0 skipped	Pipeline is robust across the entire dataset.
-----------------	-------------------	---

3.4 What Worked and What Did Not

- **What worked.** The shared preprocessing pipeline, the 12-joint graph, the augmentation strategy, and the from-scratch ST-GCN all integrate and train together cleanly, demonstrating that the core modeling approach is viable on our representation and hardware.
- **What did not (yet).** The 93% figure is training accuracy only. We have not yet created a held-out validation split, so the true generalization gap is currently unmeasured and is likely optimistic given the modest data volume. Establishing a proper split is the first item in the next phase.

4. Updated Technical Approach

Hands-on experience this phase led to several deliberate adjustments from the Milestone 1 plan. Each change is justified below:

- **Pre-training set changed from NTU/Kinetics to AIST++.** AIST++ provides clean, multi-view 3D full-body motion in a COCO format that is much closer to ARKit than either NTU RGB+D or 2D Kinetics. The smaller domain gap makes transfer to our gesture task more effective.
- **Joint subset reduced from 20–25 to 12.** ARKit has no facial or eye keypoints that correspond to COCO's head points, and a leaner reliable graph is more robust. Forcing both sources to the same 12-joint topology removes a whole class of alignment problems.
- **Stretch goal pulled forward.** Rather than wait until the end, we validated the ST-GCN now and confirmed it is feasible. The Milestone 1 classical baselines (Random Forest and SVM) and the rule-based heuristic remain planned as comparison references, not replaced.
- **Hardware adaptation.** The RTX 5070 (Blackwell architecture, compute capability sm_120) required upgrading to a CUDA 12.8 PyTorch build. This was done in an isolated environment so that the existing working setup remained intact.

5. Progress Review & Project Plan

5.1 Progress Against Milestone 1

The implemented pipeline delivers on the Milestone 1 preprocessing plan in full: root-centering,

bone-length scaling, canonical rotation, temporal resampling, and augmentation are all in place. Relative to that plan we are **ahead on modeling**, since a working ST-GCN already trains successfully, and **behind on the self-collected gesture dataset** and the real-time streaming and input layer. Those two items therefore become the central focus of the next three weeks.

5.2 Detailed Task Breakdown — Next 3 Weeks

Week	Task	Owner
Week 7	Record the five-gesture ARKit dataset (four members, varying speed, distance, and starting angle); build the gesture-embedding bank from the pre-trained backbone.	Kunal (capture), Lei (embedding)
Week 8	Create a person-split train/validation/test partition; fine-tune and evaluate gesture recognition; add the classical ML and rule-based baselines for comparison.	Lei (ML), Chenghua (metrics)
Week 9	Build the low-latency iPhone-to-PC streaming and input-emulation layer; achieve the first end-to-end live control of the Mario-style demo.	Chao (systems), all (integration)

5.3 Revised Risk Assessment

Identified Risk	Mitigation Strategy
Overfitting with no validation split	Introduce a person-split train/validation/test partition in Week 8 and report held-out accuracy rather than training accuracy.
Limited self-collected volume	Heavy augmentation is already in place, and AIST++ pre-training transfers general motion features to reduce the data requirement.
Pre-training to gesture domain gap	Shared preprocessing keeps both sources in one representation; fine-tune on ARKit gestures and compare against training from scratch.
Real-time latency from capture to input	Profile the streaming and inference loop early in Week 9, using lightweight UDP and a compact 12-joint model.

5.4 Team & Workflow

Roles remain unchanged from Milestone 1. The codebase is modular — preprocessing, augmentation, the graph definition, and the model each live in a separate file — and is shared through Git, which has allowed the data, modeling, and systems workstreams to proceed in

parallel without blocking one another.